

Information System of Research-Oriented University

OOP Final Project · Java console application

Submitted as the final deliverable of the Object-Oriented Programming course.

Built around the team's technical specification: 12 mandatory classes,
four user-facing flows, serialization, and four design patterns.

Project at a Glance

Scope

- 30 source files in 7 packages under `com.uni`
- Five user roles: Admin, Manager, Teacher, Student, ResearcherEmployee
- Two entry points:
 - `UniversityDemo` — scripted, 11 sections
 - `UniversityCLI` — interactive console UI

Tools

- Java (OpenJDK), no external libraries
- StarUML — class & use-case diagrams
- Javadoc — HTML documentation (112 files)

Spec Compliance

- All 12 mandatory classes present
- Four design patterns (spec requires ≥ 4)
- Four custom checked exceptions
- Authentication for all users
- Java object serialization with save / load
- Mark = 1st + 2nd attestation + final exam
- Credit limit (21), fail limit (3)
- News management, employee requests

Architecture

Package layout (layered, acyclic)

```
src/com/uni/
├── enums/          9 enums
├── exceptions/     4 custom exceptions
├── models/         17 classes
├── comparators/    3 Strategy implementations
├── factory/        UserFactory
├── storage/        DataStore (Singleton)
└── UniversityDemo.java
    UniversityCLI.java
```

Dependencies

enums, exceptions → models → comparators, factory, storage

Class hierarchy

```
User (abstract, Serializable)
├── Employee (abstract)
│   ├── Teacher          implements Researcher
│   ├── Manager
│   ├── Admin
│   └── ResearcherEmployee implements Researcher
└── Student              implements Researcher

Researcher (interface)
└── ResearcherDecorator  (wraps any User)
```

Four Design Patterns

Pattern	Class / Files	Role in the system
Singleton	storage/DataStore.java	Single global state holder. Thread-safe getInstance(). Serializes the entire object graph to data.ser.
Factory	factory/UserFactory.java	createUser(UserType, ...) dispatches by enum to the correct User subclass. Hides constructor variance.
Decorator	models/ResearcherDecorator.java	Wraps any User to grant the Researcher role at runtime without modifying the wrapped class. Used by Manager.makeResearcher.
Strategy	comparators/*.java (3)	DateComparator, CitationsComparator, PaperLengthComparator — interchangeable algorithms passed to printPapers(Comparator).

Each pattern closes a specific spec line

- Singleton → *"properly working serialization (think about Data Storage and some pattern)"*
- Decorator → *"Researcher created using Decorator pattern? Figure it out"*
- Strategy → *"PrintPapers(Comparator c) ... by date, citations, or length"*

Custom Exceptions

All four are **checked** (extends `Exception`) and model business-rule violations the caller must handle.

Exception	Thrown when	Spec citation
<code>CreditLimitException</code>	Student registers for a course that would exceed 21 total credits	"Students can't have more than 21 credits"
<code>MaxFailedReachedException</code>	Student has accumulated 3 failed courses; cannot register or fail more	"Students can't fail more than 3 times"
<code>LowHIndexException</code>	Supervisor candidate has h-index < 3	"If a person whose h-index < 3 is assigned as a supervisor, a custom exception must be thrown"
<code>NotAResearcherException</code>	Non-researcher tries to join a <code>ResearchProject</code>	"If someone who is not a <code>Researcher</code> tries to join the <code>ResearchProject</code> , custom <code>Exception</code> must be thrown"

All four are demonstrated in `UniversityDemo` and reachable from the CLI flows.

CLI Walkthrough · Admin & Manager

ADMIN `admin / admin`

- **1)** List users — read from DataStore
- **2)** Add student — uses UserFactory
- **3)** Remove user
- **4)** View logs — audit trail of UserActions

Closes: *"Manage Users (add, remove, update)", "See log files about user actions"*

MANAGER `manager / m1`

- **1)** Assign course to teacher
- **2)** Approve student registration
- **3)** Statistical report (graded / passed / failed / pending)
- **4)** View students
- **5)** Manage news (publish / list / pin)
- **6)** Requests (PENDING → SIGNED → APPROVED/REJECTED)
- **7)** Make researcher (Decorator)
- **8)** Enable research role (flag on Student/Teacher)

CLI Walkthrough · Teacher & Student

TEACHER `prof.smith / p1`

- **1)** View my courses
- **2)** Put mark (attempt 1 / 2 / final)
- **3)** Send complaint (UrgencyLevel)
- **4)** Send message
- **5)** View inbox
- **6)** Submit request to dean
- **7)** Research submenu

Professor => **automatically** a Researcher.

Other titles opt in via the flag.

STUDENT `alibek / s1 · year 4`

- **1)** View courses
- **2)** Register for a course `CreditLimitException`,
`MaxFailedReachedException`
- **3)** View marks
- **4)** Full transcript
- **5)** Rate teacher
- **6)** Set supervisor `LowHIndexException`
- **7)** Research submenu (if opted in)

Research Submenu (Strategy pattern in action)

Five operations

- **1)** Publish paper
- **2)** Print my papers (sortable)
- **3)** Show h-index
- **4)** All university papers (sortable)
- **5)** Top cited researcher

Available to four classes

- Teacher (Professor or opted in)
- Student (opted in)
- ResearcherEmployee (always)
- ResearcherDecorator (Manager-created)

H-index calculation

Largest h such that h papers each have $\geq h$ citations. Implementation in ResearcherSupport.

```
static double hIndex(List<ResearchPaper> p) {
    List<ResearchPaper> s = new ArrayList<>(p);
    s.sort((a, b) -> Integer.compare(
        b.getCitations(),
        a.getCitations()));

    int h = 0;
    for (int i = 0; i < s.size(); i++) {
        if (s.get(i).getCitations() >= i + 1)
            h = i + 1;
        else break;
    }
    return h;
}
```

Persistence & Demo Data

Persistence

- Java object serialization (`ObjectOutputStream`)
- Whole `DataStore` graph → `data.ser`
- All models implement `Serializable`
- Triggered by:
 - JVM shutdown hook (covers Ctrl-C, terminal close)
 - End of every user session (logout)
- Load on startup restores the entire object graph with all back-references intact

Seed dataset (out of the box)

- **11 users** — admin, manager, 4 teachers, 1 ResearcherEmployee, 4 students (years 1–4)
- **4 courses** fully staffed (CS201, MA101, HUM110, AI400)
- **9 papers** across 3 researchers (rex tops at 190 citations, prof.smith h-index = 3)
- **Marks** for two students; one fully graded, one with a pending final
- **3 news** (2 pinned, 1 with comments)
- **3 requests** across PENDING / SIGNED / APPROVED
- **1 ResearchProject** with two participants

How to Run · Deliverables

Build & run

```
mkdir -p out
javac -d out -sourcepath src \
    $(find src -name "*.java")

# scripted demo – all 11 sections
java -cp out com.uni.UniversityDemo

# interactive CLI
java -cp out com.uni.UniversityCLI
```

CLI controls

- ↑ ↓ navigate
- **Enter** select
- **0–9** digit shortcut
- **q** quit (saves state)

Deliverables in the submitted ZIP

- src/ — Java sources (30 files)
- docs/ — Javadoc HTML (112 files)
- report/report.pdf — 10-section report
- report/presentation.pdf — these slides
- StarUML .mdj — class & use-case diagrams

Spec checklist closed

- 12 mandatory classes · 4 patterns · 4 exceptions
- Serialization · Authentication · Marks/registration
- Research workflow · Supervisor with h-index check
- News · Employee requests · Manager-side reports